

CarrotDefense 项目设计文档

项目名称: CarrotDefense

技术栈: C++17 / Qt 6.11.0 Widgets / CMake 3.16+ / MSVC 2022

版本: 2026-06

一、程序功能介绍

CarrotDefense是一款基于 Qt Widgets 框架开发的 2D 塔防游戏。玩家扮演防守方，在地图路线上建造不同类型的防御塔，阻止一波波敌人到达萝卜。游戏采用 JSON 数据驱动配置，所有游戏数值均可通过修改 JSON 文件调整而无需重新编译。

1.1 核心玩法

每关开始时，玩家拥有初始金币和生命值。点击空地可从 6 种防御塔中选择建造，消耗金币。塔会自动搜索射程内的敌人并发起攻击。敌人沿预设路径前进，到达终点时扣除萝卜生命值；生命值归零则游戏失败。消灭敌人获得赏金，击败所有波次且生命值大于零即为胜利。关卡中还设有障碍物，可被攻击摧毁，获得金币并腾出空地供建造新塔。此外，点击已有塔可升级提升属性或售出回收金币，点击敌人或障碍物可将其设为优先攻击目标。

1.2 游戏流程

启动后进入主菜单，玩家可选择：

- 开始游戏：**直接加载教程关（level0），通过 7 步交互式引导学习基本操作
- 选择关卡：**进入选关界面，浏览已解锁关卡卡片，点击进入对应关卡

进入关卡后游戏自动开始，顶部 HUD 状态栏实时显示金币、生命值、当前波次、场上敌人数量。玩家可随时通过暂停按钮或 Esc 键暂停游戏。完成关卡后弹出结算界面，支持重试当前关、进入下一关或返回主菜单。通关后按列表顺序解锁下一关。

1.3 防御塔

游戏提供 6 种防御塔，定位各不相同。每座塔最高可升至 3 级，升级消耗金币并提升攻击力、射程和攻速。升级费用随等级翻倍，出售返还已投入总金币的一半。

塔类型	价格	攻击方式	定位与特点
Arrow	40g	远程单体	最基础、最廉价的塔，适合前期快速布防
Cannon	80g	远程单体	命中后产生溅射伤害，对周围敌人造成 50% 伤害，适合对付密集敌群
Ice	60g	远程单体	命中后附加减速效果，降低敌人移速至 50%，持续 2 秒；升到3级后，每颗子弹带有冰冻标记，当怪物带有三层标记便冰冻一次
Poison	65g	远程单体	命中后附加中毒效果，每秒造成 25 点伤害，持续 3 秒

塔类型	价格	攻击方式	定位与特点
Lightning	90g	远程连锁	命中后可弹跳至 3 个额外目标，适合对付分散敌人
Sun	50g	近战范围	对周围 2 格内所有敌人造成范围伤害，伤害随距离衰减

远程塔与近战塔：Arrow、Cannon、Ice、Poison、Lightning 为远程塔，每隔攻速间隔搜索射程内优先级最高的敌人，向目标发射子弹。Sun 为近战塔，不发射子弹，每隔冷却时间对周围圆形范围内的所有敌人直接造成伤害并附带效果，伤害随距离线性衰减。

Marker 效果系统：减速、中毒等功能以独立的 Marker 类实现。每种 Marker 是一个可克隆、可计时的对象。塔的 AttackResult 和 Bullet 携带 Marker 模板，命中敌人时将 Marker 克隆注入到敌人身上。敌人通过 addMarker() 接收、updateMarkers() 每帧结算。多个 Marker 可同时存在：减速和冰冻取最强因子，中毒取最高每秒伤害。效果超时自动移除。

子弹系统：每种远程塔拥有专属的子弹类，继承自 Bullet 基类，覆写 onHit() 实现各自的命中逻辑。子弹在构造时计算固定方向向量并归一化存储，飞行中不追踪目标，保持直线运动。飞出地图边界自动销毁。Cannon 子弹命中后遍历同格内所有敌人施加溅射伤害；Lightning 子弹命中后查找最近未链过的敌人依次弹跳；Ice 和 Poison 子弹命中后造成伤害并注入对应的 Marker。

属性扩展：RemoteTower 的 AttackResult 支持穿透弹（penetration，命中后继续飞行）、多发射击（shotCount）、扇形散射（spreadAngle）和波次攻击（waveCount + waveDelay，间隔发射多波子弹）等高级属性，当前版本通过 JSON 配置预留接口，部分已在 Lightning 和 Cannon 中启用。

1.4 敌人

游戏包含 5 种敌人，由工厂模式统一创建，各自拥有不同的生命值、移动速度和赏金。

敌人类型	生命值	速度	赏金	定位与特点
Normal	50	50 px/s	5	标准属性，最常见的敌人
Fast	30	80 px/s	8	移动速度快但血量低，需减速塔克制
Tank	150	30 px/s	15	血量极高但移动慢，需集中火力应对
Boss	500	25 px/s	50	极高血量，通常作为最后一波出现
Swarm	15	65 px/s	2	低血量但数量多，需溅射或范围塔应对

敌人在移动过程中可同时受到多种效果叠加：减速和冰冻取最强减速因子，中毒按最高每秒伤害生效。效果计时结束后自动移除。每个敌人头顶显示血条，颜色随血量比例变化（绿色→黄色→红色）。

1.5 波次系统

每个关卡由 JSON 文件中 waves 数组定义波次。每波包含若干条目，指定敌人类型、数量和生成间隔。首波在游戏开始 1.5 秒后生成，后续敌人按各自的间隔依次出现。每波所有敌人被消灭后发放波次通关奖励（基础奖金 + 波次号 × 增量），并自动进入下一波。

1.6 障碍物

地图上预设 Tree 和 Statue 两种障碍物，各占据若干格子。玩家可点击障碍物将其设为塔的优先攻击目标。障碍物被摧毁后占据的格子变为可用塔位，同时获得金币奖励。Statue 比 Tree 拥有更高的生命值和赏金。

1.7 关卡系统

关卡通过 `config/levels.json` 文件索引管理。每个条目包含关卡 ID、名称和 JSON 文件路径。每关的独立 JSON 文件中定义地图尺寸、路径坐标、波次配置、初始金币和生命值。Debug 构建下所有关卡默认解锁；Release 构建下初始仅教程关解锁，通关后按列表顺序解锁后续关卡，支持非连续关卡 ID。

1.8 界面与交互

游戏界面由三页组成：主菜单、选关界面、游戏画面。游戏画面顶部显示半透明 HUD 状态栏，包含暂停按钮和实时数据（金币、生命、波次、敌人数量）。点击空地弹出塔选择弹窗，展示 6 种塔的按钮及价格；金币不足时按钮自动置灰。点击已有塔弹出详情面板，显示等级、伤害和射程，hover Upgrade 按钮可预览升级后的塔贴图和射程圈，hover Sell 按钮高亮目标格红色警示框。点击敌人或障碍物可将其设为优先攻击目标，目标上方显示十字准心指示器。Esc 键或 HUD 暂停键弹出暂停遮罩，经二级确认后退出到选关。

1.9 新手教程

玩家首次进入游戏自动加载教程关。教程通过半透明遮罩、挖洞高亮框、金色箭头和底部文字提示，分为 7 个步骤依次引导玩家：了解游戏目标 → 认识 HUD 信息区域 → 放置第一座防御塔 → 观察塔自动攻击敌人 → 体验升级塔 → 锁定并攻击障碍物。教程结束后恢复游戏正常运行。教程使用事件过滤器接管鼠标操作，通过冻结游戏时间（`timeScale=0`）实现暂停但不锁定输入。

1.10 自适应与扩展

游戏内所有实体采用网格坐标存储位置和移动，渲染时统一转换为像素坐标。窗口缩放时仅更新网格大小和偏移量两个参数，无需同步任何实体状态。游戏配置全部由 JSON 文件驱动，新增关卡只需添加 JSON 文件并在索引中注册，无需修改代码。

二、项目各模块与类设计细节

2.1 整体架构

项目采用严格的三层解耦架构。UI 层（`ui/`）持有游戏逻辑层（`game/`）对象的指针并通过 `#include` 使用游戏类型。游戏逻辑层通过 `DataManager` 单例读取配置层（`config/`）的 JSON 数据。核心约束为 `game/` 永不 `#include` 任何 `ui/` 头文件，保证逻辑层可独立于界面进行单元测试。

`main.cpp` 启动时按顺序加载：`shared.json`（塔与敌人的全局属性，加载前清空旧哈希表）→ `levels.json`（关卡索引）→ 索引中首个关卡的地图数据，最后创建 `Mainwindow` 进入主菜单。

`Mainwindow` 通过 `QStackedWidget` 管理三页：`MainMenuWidget`（主菜单，idx 0）、`LevelSelectWidget`（选关，idx 1）、`GameScene`（游戏画面，idx 2）。游戏结果通过动态创建 `GameResultWidget` 展示，`cleanupOverlay()` 在创建前清理旧实例。`MainWindow` 已移除原有的 `ToolBar` 和 `StatusBar`。

2.2 GameScene 子系统

GameScene 是游戏页面的核心容器，构造函数中创建 10 个子系统并建立信号连接：

子系统	类型	职责
m_spatialGrid	SpatialGrid	15×12 地图网格。存储路径信息 (m_isPath)、障碍物占用 (m_obstacleCell)、每格实体列表 (m_entityGrid)。提供 gridToPixel / pixelToGrid 坐标转换、syncEntityGrid() 实体网格同步。格子中心采用 (N+0.5, N+0.5) 约定。
m_gameController	GameController	游戏逻辑主循环，QObject。每帧 update(dt, towers) 驱动敌人移动、塔攻击、子弹飞行、碰撞检测和波次推进。持有 m_timescale 变量 (默认 1.0)，教程中设为 0.0 冻结世界同时保持 isPaused()=false。
m_towerManager	TowerManager	塔的增删查，QObject。维护 m_towers 列表。canPlaceTower() 检查格子合法性，placeTowerAt() 完成扣费与放置，发射 towersChanged 触发渲染更新和教程检测。
m_gameRenderer	GameRenderer	游戏内视觉绘制，QWidget。paintEvent 按 9 层顺序绘制：背景 → 网格 (草/路贴图) → 路径线 → 障碍物 → 塔 (贴图+fallback 几何体) → 敌人 (贴图+血条+状态色叠加) → 子弹 (贴图+方向旋转) → 交互预览 (选中高亮/hover 预览/射程圈) → 游戏结束遮罩。
m_inputHandler	InputHandler	鼠标和键盘输入处理，非 QObject。点击优先级：运行检查 → 弹窗关闭 → 锁敌 → 放塔。鼠标移动更新 hover 坐标和范围预览。
m_panelController	PanelController	面板协调中心，QObject。管理 TowerPanel 和 TowerSelectionPopup 的显示/隐藏、位置计算、升级/出售/选塔完整逻辑。发射 towerPanelShown/Hidden 驱动预览效果。
m_towerPanel	TowerPanel	塔详情面板，QWidget + .ui。显示等级/名称/伤害/射程。Upgrade 根据金币动态启用，满级显示 MAX。Sell 显示出售价格。
m_selectionPopup	TowerSelectionPopup	塔选择弹窗，QWidget。2×3 网格 6 种塔按钮。updateGold() 使金币不足的按钮自动置灰。
m_gameHUD	GameHUD	顶部状态栏，QWidget + .ui，MVC Dumb View。零 game/ 依赖，纯 int setter 接口 (setGold/setLives/setWave/setEnemies/setPausedState)。左侧暂停按钮，右侧 4 组图标占位+数值标签。
m_overlay	GameOverlay	暂停/确认/消息遮罩，QWidget + .ui。QStackedWidget 三页，paintEvent 半透明黑底，event() 拦截鼠标防止穿透。

2.3 游戏逻辑层详细设计

GameController 主循环。每帧 update(dt, towers) 执行 15 步：(1) 波次计时 → (2) 出怪 → (3) 敌人移动 → (4) 扣血检查 → (5) 塔攻击决策 → (6) RemoteTower 创建子弹 → (7) MeleeTower 范围结算 → (8) 实体网格同步 → (9) 子弹移动+碰撞 → (10) 障碍物动画 → (11) 命中处理 → (12) 清理死亡实体 → (13) 波次完成奖励 → (14) 失败判断 → (15) emit statsChanged。dt *= m_timescale，dt <= 0 时跳过以支持教程冻结。

WaveManager 波次管理。m_spawnQueue 存储 vector<pair<EnemyType, double>> (类型+间隔)。nextWave() 从 DataManager 读取当前关卡波次压入队列。popSpawnType() 使用队列中的 interval 替代硬编码 0.8 秒。reset() 末尾调用 buildwaves() 重新拉取当前关卡数据。

SpatialGrid 空间索引。syncEntityGrid() 每帧清空后按 floor(gridPos) 将活跃实体分格。子弹碰撞只需查所在格内实体，从 O(n²) 降为 O(n)。

网格坐标系。所有实体采用网格坐标存储，渲染时经 cellSize/offset 转像素。格子中心 (N+0.5, N+0.5)，gridToPixel 返回像素中心，pixelToGrid 返回浮点坐标。resize 时仅更新 cellSize/offset，无需同步实体。

Marker 多态效果系统。Marker 抽象基类定义 clone() 和 update()。SlowMarker 降低速度，PoisonMarker 持续伤害，FreezeMarker 冻结。Enemy 通过 addMarker() 添加，updateMarkers() 逐帧结算。Tower 和 Bullet 通过 m_markerTemplates 携带模板并克隆注入。多 Marker 共存，减速取最强因子，中毒取最高 dps。

2.4 核心信号链

- 游戏状态： GameController::statsChanged → GameScene::onStatsChanged → GameHUD 各 setter + PanelController::onStatsChanged （面板金币检测）
- 塔放置： TowerSelectionPopup::towerSelected → PanelController → TowerManager::placeTowerAt → towersChanged → GameRenderer::update
- 升级： TowerPanel::upgradeClicked → PanelController::onUpgradeClicked → tower->upgrade() → statsChanged
- 暂停： GameHUD::pauseClicked → showPauseOverlay → GameOverlay::continueClicked → resumeGame
- 教程： ClickContinue 通过 Arrow 信号， PlaceTower/TowerUpgraded 通过信号检测， ClickCell 通过 eventFilter 坐标检测， EnemyKilled 通过回调

2.5 实体体系

类别	基类	子类
敌人	Enemy	NormalEnemy(50hp) / FastEnemy(30hp) / TankEnemy(150hp) / BossEnemy(500hp) / SwarmEnemy(15hp)
塔	Tower → RemoteTower / MeleeTower	Arrow(40g) / Cannon(80g,溅射) / Ice(60g,减速) / Poison(65g,中毒) / Lightning(90g,连锁) / Sun(50g,近战)
子弹	Bullet	ArrowBullet / CannonBullet / IceBullet / PoisonBullet / LightningBullet
障碍物	Obstacle	TreeObstacle(100hp) / StatueObstacle(200hp)
效果	Marker	SlowMarker / PoisonMarker / FreezeMarker

所有实体采用"基类抽象 + 工厂创建"模式。Enemy 子类覆写 getStats() 返回属性；Tower 子类覆写 drawBody() 负责外观；Bullet 子类覆写 onHit() 执行命中逻辑；Obstacle 子类覆写 drawBody()。创建通过 Factory 函数根据枚举返回 unique_ptr。

2.6 教程系统设计

教程以 level0 独立关卡存在。TutorialController 为 QObject 子类，维护 7 个步骤的状态机，通过 installEventFilter(this) 安装在 GameScene 上接管鼠标事件。TutorialArrow 为 QWidget 子类，通过 setMask(整屏 - 目标格) 实现挖洞遮罩：目标格外点击被 mousePressEvent::accept() 拦截，目标格内点击因 mask 排除而穿透至下层。

七步完成条件分别采用不同检测机制：ClickContinue 通过 Arrow 的 `clicked()` 信号；PlaceTower 连接 `TowerManager::towersChanged` 检测目标格有塔；TowerUpgraded 连接 `GameController::statsChanged` 检测塔等级 ≥ 2 ；EnemyKilled 通过 `onEnemyKilled` 回调；ClickCell 通过 `eventFilter` 坐标检测 + `QTimer::singleShot(0)` 延迟推进。教程通过 `GameController::setTimeScale(0)` 冻结时间。步骤 4 时调用 `GameScene::resumeClock()` 启动游戏循环。结束后移除 `eventFilter`、恢复 `timeScale`。

2.7 数据流与存储

`DataManager` 为单一数据访问点，Meyers 单例。`loadShared()` 加载塔/敌人/障碍物属性，加载前清空旧哈希表。`loadLevel()` 加载关卡的 `MapData`、`WaveEntry` 列表和 `ObstacleEntry` 列表。`loadLevelsIndex()` 加载关卡目录。贴图通过 `getTexture(path)` 加载，内部 `QHash<QString, QPixmap>` 缓存。`assets/` 由 CMake `POST_BUILD` 复制到可执行文件目录。

三、小组成员分工

成员	负责范围	主要工作
盛铖开 (SmallSck)	UI 框架 / 关卡系统 / JSON 配置 / 教程	MainWindow 页面导航与关卡解锁逻辑；GameController 主循环框架实现；GameOverlay 和 GameHUD 制作；JSON关卡加载系统数据驱动改造；新手教程系统的架构设计与实现；
刘张懿 (n0v5)	视觉渲染 / UX 交互 / 面板	塔、敌人、子弹、地面等外部贴图系统的引入与渲染；GameRenderer 选中高亮、hover 预览、射程圈等视觉特效；TowerPanel 和 TowerSelectionPopup 的 UX 增强；平台图标设计
王千一 (wqy-lab)	游戏战斗逻辑 / 实体体系	游戏内部战斗和索敌逻辑；SpatialGrid 网格坐标系统；Bullet 多态与 Marker 多态效果系统；Tower 基类与 RemoteTower 攻击属性扩展；Obstacle障碍物设计

四、项目总结与反思

4.1 成果

本项目在 Qt Widgets 框架下实现了一个功能完整、架构清晰的塔防游戏。主要成果包括：完整游戏闭环体验；数据驱动的 JSON 配置体系使关卡数据可脱离代码修改；可扩展的实体工厂体系组织 6 种塔、5 种敌人、5 种子弹、3 种 Marker；稳定的 MVC 分层架构（GameHUD 零 game/ 依赖，教程系统零侵入集成）；增量式代码质量提升（约 120 行冗余代码清理、硬编码路径数据驱动改造、编译期安全语法迁移、真实时间步长替换硬编码 dt）；约 600 行独立教程系统实现 7 步交互式教学。

4.2 经验

JSON 配置化带来的迭代效率远超界面控件层面调整；eventFilter 是实现 UI 解耦的关键工具；多人协作中明确模块边界和接口约定是避免冲突的前提；面对技术债务直接剔除重写比持续打补丁更利于长期维护。

4.3 待改进

贴图资源管理可抽取 AssetManager 统一管理路径与缓存；面板 FocusOut 双路径冲突可改用 Qt::Tool 解决；游戏逻辑层需补充自动化测试；贴图文件可迁移至 Qt Resource System 简化部署。